

08 · Subqueries

Why this matters in interviews. "Second highest salary", "employees earning more than their department average", "IN vs EXISTS", "why does NOT IN return nothing?", "what does > ANY mean?" — subqueries are where interviewers separate people who *memorised* SQL from people who understand it. Several of these have classic traps; each one is demonstrated below with real data.

In this chapter: what a subquery is · the four shapes (scalar, row, column, table) · where they can appear · correlated subqueries · EXISTS · IN / NOT IN and the NULL trap · ANY / ALL / SOME · subquery vs join · classic patterns.

Sample tables (see **00 · Start Here**):

dept

dept_id	dept_name	location
10	Management	Chennai
20	Engineering	Bengaluru
30	Sales	Mumbai
40	HR	Pune

emp

emp_id	emp_name	job	manager_id	salary	comm	dept_id	hire_date
101	Asha	CEO	NULL	90000	NULL	10	2015-01-10
102	Bala	Manager	101	70000	NULL	20	2016-03-15
103	Chitra	Developer	102	50000	NULL	20	2018-07-01
104	Dinesh	Developer	102	50000	NULL	20	2019-02-20
105	Esha	Tester	102	40000	NULL	20	2020-11-05
106	Farhan	Manager	101	65000	NULL	30	2017-05-30
107	Gita	Sales Rep	106	35000	5000	30	2021-01-12
108	Hari	Sales Rep	106	35000	3000	30	2022-06-18
109	Isha	Intern	103	20000	NULL	20	2024-01-08
110	Jai	Consultant	101	45000	NULL	NULL	2023-04-01

8.1 What is a subquery?

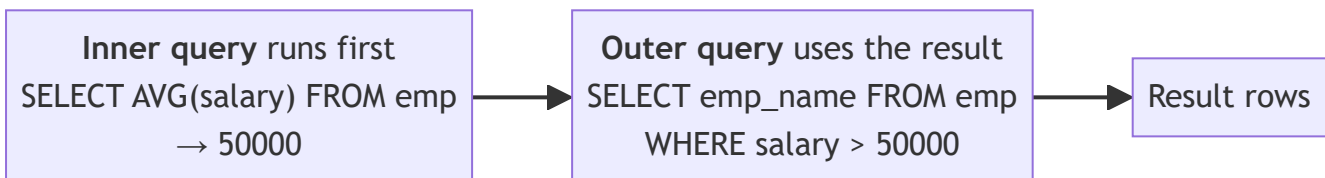
A **subquery** (also called an **inner query** or **nested query**) is a **query inside another query**. The outer query uses the inner query's result. Subqueries are used to:

- **filter** data,
- **compute** values,
- **compare** results,
- **use intermediate results**.

```
SELECT column1
FROM table1
WHERE column1 = (SELECT column2 FROM table2 WHERE condition);
```

Rules

Rule	Detail
Always in parentheses	(SELECT ...)
A scalar subquery must return only one value	more than one row → ORA-01427: single-row subquery returns more than one row
Zero rows	a scalar subquery then evaluates to NULL (no error)
Can be nested multiple levels deep	but readability drops fast
Allowed in SELECT , FROM , WHERE and HAVING (and in INSERT , UPDATE , DELETE)	see §8.3



8.2 The four shapes of a subquery

What matters is the **shape of the result** the inner query returns — it decides which operators you may put in front of it.

Type	Returns	Typical operators
Scalar	one row, one column — a single value	= , > , < , ...
Row	one row , several columns	(a, b) = (...)
Column	one column , several rows	IN , NOT IN , ANY , ALL
Table	several rows and columns — a full result set	used in FROM (derived table / inline view)

Scalar — one value

Employees paid more than the company average.

```
SELECT emp_name, salary
FROM emp
WHERE salary > (SELECT AVG(salary) FROM emp)
ORDER BY salary DESC;
```

Result

emp_name	salary
Asha	90000
Bala	70000
Farhan	65000

The inner query returns `50000` (a single value); the outer one compares against it.

Column — one column, many rows

Employees who work in departments located in Chennai or Mumbai.

```
SELECT emp_name, dept_id
FROM emp
WHERE dept_id IN (SELECT dept_id FROM dept WHERE location IN ('Chennai', 'Mumbai'))
ORDER BY emp_id;
```

Result

emp_name	dept_id
Asha	10
Farhan	30
Gita	30
Hari	30

The inner query returns the list `(10, 30)`; `IN` tests membership.

Row — one row, several columns

The best-paid employee in each department (compare a pair of columns against a list of pairs).

```
SELECT emp_name, dept_id, salary
FROM emp
WHERE (dept_id, salary) IN (SELECT dept_id, MAX(salary) FROM emp GROUP BY dept_id)
ORDER BY dept_id;
```

Result

emp_name	dept_id	salary
Asha	10	90000
Bala	20	70000
Farhan	30	65000

Trap — a widely copied wrong version. `WHERE (Department, Salary) = (SELECT Department, MAX(Salary) FROM Employees)` does **not** work: mixing a plain column with `MAX()` and no `GROUP BY` raises `ORA-00937: not a single-group group function`, and even with a `GROUP BY` the inner query returns **one row per department**, so `=` (which needs exactly one row) is the wrong operator. Use `IN` with a `GROUP BY` as above. *(Jai has no department, so his `NULL` pair never matches — he is not in the result.)*

Table — a whole result set used like a table

Treat the high earners as a temporary table:

```
SELECT *  
FROM (SELECT emp_name, salary FROM emp WHERE salary > 50000) high_earners  
ORDER BY salary DESC;
```

Result

emp_name	salary
Asha	90000
Bala	70000
Farhan	65000

`high_earners` is a **derived table** (inline view) — a temporary result the outer query can join, filter and sort.

8.3 Where a subquery can appear

In the `WHERE` clause — filter by a computed value

```
SELECT Name FROM Employees  
WHERE DepartmentID = (SELECT ID FROM Departments WHERE Name = 'HR');    -- employees in HR
```

In the `FROM` clause — a temporary table

```
SELECT * FROM (SELECT Name, Salary FROM Employees WHERE Salary > 50000) HighEarnings;
```

In the `SELECT` clause — a value computed per row

A **scalar subquery** in the select list runs for each output row:

```
SELECT emp_name,  
       (SELECT dept_name FROM dept WHERE dept_id = e.dept_id) AS dept  
FROM emp e  
ORDER BY emp_id;
```

Result

emp_name	dept
Asha	Management
Bala	Engineering
Chitra	Engineering
Dinesh	Engineering
Esha	Engineering
Farhan	Sales
Gita	Sales
Hari	Sales
Isha	Engineering
Jai	NULL

Show employee names along with their department name using a subquery — it works like a join, but Jai (no department) gets `NULL` rather than disappearing.

In the `HAVING` clause — compare a group to an overall figure

Departments whose average salary is above the company average:

```
SELECT dept_id, ROUND(AVG(salary)) AS avg_salary
FROM emp
GROUP BY dept_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM emp)
ORDER BY dept_id;
```

Result

dept_id	avg_salary
10	90000

8.4 Correlated subqueries

A **correlated subquery** depends on values from the **outer query**. Unlike a plain (simple) subquery — which can be run on its own — a correlated one **cannot be executed alone**, because it references columns of the outer query. Conceptually it is **evaluated once for each row** the outer query processes.

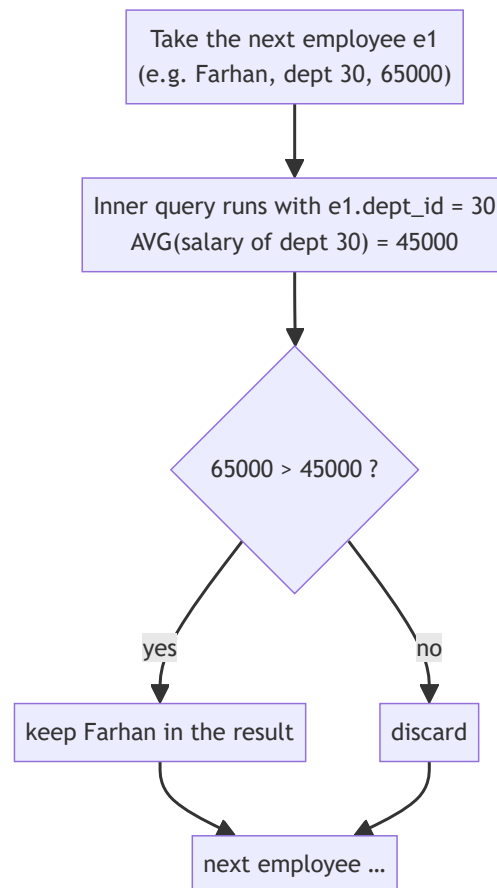
Employees who earn more than the average of their own department:

```
SELECT e1.emp_name, e1.dept_id, e1.salary
FROM emp e1
WHERE e1.salary > (SELECT AVG(e2.salary)
                  FROM emp e2
                  WHERE e2.dept_id = e1.dept_id)
ORDER BY e1.emp_id;
```

Result

emp_name	dept_id	salary
Bala	20	70000
Chitra	20	50000
Dinesh	20	50000
Farhan	30	65000

For each employee `e1`, the inner query calculates the average salary of employees in the **same department** (`e2.dept_id = e1.dept_id`), then checks whether `e1.salary` is greater than that average.



Department averages: 10 → 90,000 · 20 → 46,000 · 30 → 45,000. So **Bala** (70,000), **Chitra** and **Dinesh** (50,000) in department 20, and **Farhan** (65,000) in department 30 qualify. Asha does not — she is her department's average, and 90,000 is not *greater* than itself — and Jai has no department, so his subquery is `NULL` and he never qualifies.

Performance. A naive reading suggests one inner query *per row*; the optimizer often rewrites it as a join, but on large tables a correlated subquery can be slow. The same question is often faster with a **window function** (Chapter 11): `AVG(salary) OVER (PARTITION BY dept_id)`.

Correlated `SELECT` -list subquery — each employee with the number of people in their department:

```

SELECT emp_name, dept_id,
       (SELECT COUNT(*) FROM emp x WHERE x.dept_id = e.dept_id) AS dept_size
FROM   emp e
ORDER BY emp_id;

```

Result

emp_name	dept_id	dept_size
Asha	10	1
Bala	20	5
Chitra	20	5
Dinesh	20	5
Esha	20	5
Farhan	30	3
Gita	30	3
Hari	30	3
Isha	20	5
Jai	NULL	0

8.5 EXISTS and NOT EXISTS

EXISTS tests for the existence of any record in a subquery. It returns **TRUE** if the subquery returns one or more rows.

```

SELECT column_name(s)
FROM   table_name
WHERE  EXISTS (SELECT column_name FROM table_name WHERE condition);

```

Departments that have at least one employee:

```

SELECT d.dept_name
FROM   dept d
WHERE  EXISTS (SELECT 1 FROM emp e WHERE e.dept_id = d.dept_id)
ORDER BY d.dept_name;

```

Result

dept_name
Engineering
Management
Sales

Departments with no employees:

```
SELECT d.dept_name
FROM   dept d
WHERE  NOT EXISTS (SELECT 1 FROM emp e WHERE e.dept_id = d.dept_id);
```

Result

dept_name
HR

- `EXISTS` only asks "*is there a row?*" — **what** the subquery selects is irrelevant, so `SELECT 1` (or `SELECT *`) is conventional.
- It **stops at the first match**, which is why it is often fast for existence checks.
- It is **NULL-safe** — unlike `NOT IN`, as the next section proves.

8.6 `IN` and `NOT IN` — and the NULL trap

`IN` (subquery) returns the rows of the outer query whose value **appears in the subquery's result**; `NOT IN` the ones that do not.

```
SELECT emp_name, dept_id
FROM   emp
WHERE  dept_id IN (SELECT dept_id FROM dept WHERE location = 'Bengaluru');
```

Result

emp_name	dept_id
Bala	20
Chitra	20
Dinesh	20
Esha	20
Isha	20

The natural attempt to find departments with no employees:

```
SELECT dept_name
FROM   dept
WHERE  dept_id NOT IN (SELECT dept_id FROM emp);
```

Result

dept_name
(no rows)

No rows — wrong! HR (dept 40) has no employees, so it should appear. The cause: the subquery's list is (20, 30, 10, NULL, ...) because **Jai's dept_id is NULL**. `dept_id NOT IN (... , NULL)` expands to `dept_id <> 10 AND dept_id <> 20 AND dept_id <> 30 AND dept_id <> NULL` — and `dept_id <> NULL` is **UNKNOWN** for every row,

so the whole `AND` can never be `TRUE`.

The single most important `NOT IN` rule. If the subquery can return **even one `NULL`**, `NOT IN` returns **no rows at all**. Three fixes:

```
SELECT dept_name FROM dept
WHERE dept_id NOT IN (SELECT dept_id FROM emp WHERE dept_id IS NOT NULL);  -- 1) filter the NULLs out
```

Result

dept_name
HR

```
SELECT d.dept_name FROM dept d
WHERE NOT EXISTS (SELECT 1 FROM emp e WHERE e.dept_id = d.dept_id);      -- 2) NOT EXISTS (best)
```

Result

dept_name
HR

```
SELECT d.dept_name FROM dept d
LEFT JOIN emp e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;                                                  -- 3) LEFT JOIN anti-join
```

Result

dept_name
HR

	IN / NOT IN	EXISTS / NOT EXISTS
Question asked	is the value in the list?	is there any row ?
NULLs in the subquery	<code>IN</code> ignores them; <code>NOT IN</code> breaks	no problem
Multi-column match	awkward	easy (correlate on several columns)
Typical best use	small, fixed, non-NULL lists	large or possibly-NULL sets; existence tests

(The same trap appears in the textbook example "customers who have not placed any orders": `WHERE CustomerID NOT IN (SELECT CustomerID FROM Orders)` works only while `Orders.CustomerID` never contains `NULL`.)

8.7 ANY, ALL and SOME

ANY and ALL compare a value to the set of values returned by a subquery. They are used together with a comparison operator (= , != , > , < , >= , <=), and the subquery must return a single column.

Keyword	The condition is TRUE if it holds for ...
ANY (or SOME)	at least one of the values returned
ALL	every value returned

```
SELECT column_name FROM table_name WHERE column OPERATOR ANY (subquery);  
SELECT column_name FROM table_name WHERE column OPERATOR ALL (subquery);
```

The easiest way to remember them is that they are shorthand for MIN / MAX :

Expression	Means the same as
x > ANY (list)	x > MIN(list) — greater than at least one
x < ANY (list)	x < MAX(list)
x = ANY (list)	x IN (list)
x > ALL (list)	x > MAX(list) — greater than every one
x < ALL (list)	x < MIN(list)
x != ALL (list)	x NOT IN (list)
x != ANY (list)	at least one differs — not NOT IN !

Tables used (from the course):

products

product_id	name	price
1	Pen	10
2	Pencil	5
3	Eraser	7
4	Marker	15
5	Scale	8

orders

order_id	product_id	quantity
101	1	3
102	2	1
103	3	5

> ANY

Products priced above at least one of Pencil (5) or Eraser (7).

```
SELECT product_id, name, price
FROM products
WHERE price > ANY (SELECT price FROM products WHERE name IN ('Pencil', 'Eraser'));
```

Result

product_id	name	price
1	Pen	10
3	Eraser	7
4	Marker	15
5	Scale	8

This means `price > 5 OR price > 7`, i.e. `price > 5` — so **Eraser (7)** qualifies too, because $7 > 5$. Four products: Pen 10, Eraser 7, Marker 15, Scale 8. (Only Pencil, at 5 itself, is excluded.)

`= ANY (the same as IN)`

Orders for products priced below 10. The prices below 10 belong to Pencil (5), Eraser (7) and Scale (8):

```
SELECT *
FROM orders
WHERE product_id = ANY (SELECT product_id FROM products WHERE price < 10);
```

Result

order_id	product_id	quantity
102	2	1
103	3	5

`< ANY`

Products cheaper than the Marker (15) — everything else:

```
SELECT product_id, name, price
FROM products
WHERE price < ANY (SELECT price FROM products WHERE name = 'Marker');
```

Result

product_id	name	price
1	Pen	10
2	Pencil	5
3	Eraser	7
5	Scale	8

`!= ANY` **is not** `NOT IN`

`!= ANY (list)` is TRUE when the value differs from **at least one** value in the list. Take the prices below 10: (5, 7, 8) .

```
SELECT product_id, name, price
FROM   products p
WHERE  price != ANY (SELECT price FROM products WHERE price < 10);
```

Result

product_id	name	price
1	Pen	10
2	Pencil	5
3	Eraser	7
4	Marker	15
5	Scale	8

Every product is returned — even Pencil (5), because 5 differs from 7 (and from 8). With two or more *different* values in the list, `!= ANY` is true for every row, which makes it almost useless. What people usually mean is `NOT IN` , a.k.a. `!= ALL` :

```
SELECT product_id, name, price
FROM   products
WHERE  price != ALL (SELECT price FROM products WHERE price < 10);
```

Result

product_id	name	price
1	Pen	10
4	Marker	15

Only **Pen (10)** and **Marker (15)** — the prices *not* in {5, 7, 8} .

Trap. A very common textbook line says " `!= ANY` returns rows where the price is not equal to 5 OR 7 OR 8, so prices 10 and 15 are returned". That reading is **wrong**: it treats `!= ANY` as `NOT IN` . Pencil (5) *is* returned by `!= ANY` , as shown above.

> ALL

Products priced above every one of Pencil (5), Eraser (7) and Scale (8) → `price > 5 AND price > 7 AND price > 8` → `price > 8` :

```
SELECT product_id, name, price
FROM   products
WHERE  price > ALL (SELECT price FROM products WHERE name IN ('Pencil', 'Eraser', 'Scale'));
```

Result

product_id	name	price
1	Pen	10
4	Marker	15

Only Pen (10) and Marker (15).

< ALL

Products cheaper than both the Pen (10) and the Marker (15) → `price < 10 AND price < 15` → `price < 10`:

```
SELECT product_id, name, price
FROM   products
WHERE  price < ALL (SELECT price FROM products WHERE name IN ('Pen', 'Marker'));
```

Result

product_id	name	price
2	Pencil	5
3	Eraser	7
5	Scale	8

Pencil 5, Eraser 7, Scale 8.

= ALL

Usually the subquery returns just one value, so `= ALL` collapses to plain `=`. *Products priced the same as the Pencil:*
`WHERE price = ALL (SELECT price FROM products WHERE name IN ('Pencil'))` → Pencil (5).

Notes and edge cases

- If the subquery returns **no rows**, **ALL is always TRUE** (nothing can contradict it) and **ANY is FALSE**.
- If the subquery contains a **NULL**, a comparison with `ALL` is not TRUE (UNKNOWN) unless the NULLs are filtered out — the same disease as `NOT IN`.
- `ALL` and `ANY` are used **with subqueries**, not with fixed value lists (for a list use `IN`, or `> ANY (5, 7)` in Oracle's literal-list form).

Summary

Operator	Result is TRUE if the value is ...
<code>> ANY</code>	greater than at least one in the list
<code>< ANY</code>	less than at least one
<code>= ANY</code>	equal to any one (<code>IN</code>)
<code>!= ANY</code>	not equal to at least one
<code>> ALL</code>	greater than every value
<code>< ALL</code>	less than every value
<code>= ALL</code>	equal to all values (usually one value)

Operator	Result is TRUE if the value is ...
<code>!= ALL</code>	not equal to any of them (<code>NOT IN</code>)

A worked use from the practice set: *female project leaders who earn more than every male project leader* — `salary > ALL (SELECT salary FROM employee WHERE gender = 'M' AND designation = 'Project Leader')`. It's equivalent to `> (SELECT MAX(...))`.

8.8 Subquery or join?

	Join	Subquery
Returns columns from	both tables	usually just the outer table
Readability	great for combining data	great for "compute X, then filter by it"
Existence tests	needs care (duplicates!)	<code>EXISTS</code> is natural
Aggregate comparison	needs a derived table	<code>> (SELECT AVG(...))</code> is natural
Performance	usually fine; optimizers love joins	often rewritten into a join internally

The same request both ways — *departments that have employees*:

```
-- subquery                                -- join (needs DISTINCT, or you get one row per
employee!)
SELECT dept_name FROM dept d                SELECT DISTINCT d.dept_name
WHERE EXISTS (SELECT 1 FROM emp e           FROM dept d JOIN emp e ON e.dept_id = d.dept_id;
            WHERE e.dept_id = d.dept_id);
```

8.9 Classic patterns

The second-highest salary

The highest salary *below* the maximum:

```
SELECT MAX(salary) AS second_highest
FROM emp
WHERE salary < (SELECT MAX(salary) FROM emp);
```

Result

second_highest
70000

(The *N*-th highest with ties handled properly is a `DENSE_RANK()` job — Chapters 11 and 16.)

Highest paid in each department (correlated)

```
SELECT d.dept_name, e.emp_name, e.salary
FROM   emp e
JOIN    dept d ON e.dept_id = d.dept_id
WHERE   e.salary = (SELECT MAX(salary) FROM emp WHERE dept_id = e.dept_id)
ORDER  BY d.dept_name;
```

Result

dept_name	emp_name	salary
Engineering	Bala	70000
Management	Asha	90000
Sales	Farhan	65000

Department with the most people in one job (subquery inside HAVING)

Which department has the most Developers? Count developers per department, keep the group(s) whose count equals the maximum of those counts:

```
SELECT d.dept_name, COUNT(*) AS developers
FROM   emp e
JOIN    dept d ON e.dept_id = d.dept_id
WHERE   e.job = 'Developer'
GROUP  BY d.dept_name
HAVING COUNT(*) = (SELECT MAX(cnt)
                   FROM (SELECT COUNT(*) AS cnt
                         FROM   emp
                         WHERE  job = 'Developer'
                         GROUP  BY dept_id));
```

Result

dept_name	developers
Engineering	2

A subquery **inside a subquery** (the derived table `(SELECT COUNT(*) ... GROUP BY dept_id)`), feeding a scalar `MAX` into `HAVING`.

Employees earning more than their manager

Self join — or a correlated subquery:

```
SELECT e.emp_name
FROM   emp e
WHERE  e.salary > (SELECT m.salary FROM emp m WHERE m.emp_id = e.manager_id);
```

Interview corner

Q. What is a subquery? Types? A query nested in another. By result shape: scalar, row, column, table (derived). By dependency: simple vs correlated.

Q. Correlated vs non-correlated? A non-correlated subquery runs independently and once; a correlated one references the outer query and is conceptually evaluated per outer row.

Q. IN vs EXISTS ? `IN` compares a value against a list; `EXISTS` checks whether any row is returned. `EXISTS` short-circuits and is NULL-safe; prefer it for large or nullable sets.

Q. Why does `NOT IN` return nothing when the subquery contains NULL? `x <> NULL` is UNKNOWN, so the `AND` chain can never be TRUE. Use `NOT EXISTS`, filter `WHERE col IS NOT NULL`, or `LEFT JOIN ... IS NULL`.

Q. `> ANY` vs `> ALL` ? `> ANY` = greater than the minimum of the list; `> ALL` = greater than the maximum.

Q. `!= ANY` vs `NOT IN` ? Not the same. `!= ANY` is true when the value differs from at least one item — nearly always TRUE. `NOT IN` is `!= ALL`.

Q. What happens if a scalar subquery returns more than one row? Oracle raises `ORA-01427`. (Zero rows → NULL.)

Q. Find the second-highest salary without `LIMIT / TOP`. `SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp)`.

Q. Can a subquery be in `FROM` ? What is it called? Yes — a derived table / inline view. (Oracle doesn't require an alias, MySQL and PostgreSQL do; always give one.)

Q. Which is faster, a join or a subquery? Depends; modern optimizers often turn one into the other. Choose the clearer form, then check the execution plan.

Quick recap

- A **subquery** is a query in parentheses inside another; shapes: **scalar** (1 value), **row**, **column** (list), **table** (derived).
- Allowed in `SELECT`, `FROM`, `WHERE`, `HAVING`. A scalar subquery returning >1 row ⇒ `ORA-01427`.
- **Correlated** = references the outer query, evaluated per row (e.g. above own-department average).
- `EXISTS` = "any row?" — short-circuits, NULL-safe. **`NOT IN` + a NULL in the list ⇒ no rows**; use `NOT EXISTS`.
- `> ANY` = `> MIN`; `> ALL` = `> MAX`; `= ANY` = `IN`; `!= ALL` = `NOT IN`; `!= ANY` ≠ `NOT IN`. Empty subquery: `ALL` TRUE, `ANY` FALSE.
- Patterns: second-highest = `MAX(x) WHERE x < (SELECT MAX(x))`; top per group = correlated = `(SELECT MAX ...)`.